

Segurança tecnológica



s **fundamentos de segurança da informação** visam proteger a integridade, confidencialidade e disponibilidade das informações dentro de uma organização. Esses princípios são essenciais para prevenir acesso não autorizado, perda de dados e outras ameaças.

1. Confidencialidade

- Garante que a informação seja acessada apenas por pessoas autorizadas. Isso inclui criptografar dados e implementar controle de acesso para proteger informações sensíveis.

2. Integridade

- Assegura que as informações sejam mantidas precisas e completas, sem alterações não autorizadas. Mecanismos como assinaturas digitais e hashes são usados para verificar a integridade dos dados.

3. Disponibilidade

- Assegura que a informação esteja acessível quando necessário. Isso envolve garantir que sistemas e redes estejam operacionais e funcionando corretamente, além de prevenir ataques como DoS (Denial of Service).

4. Autenticidade

- Verifica a identidade de usuários ou sistemas para garantir que as partes envolvidas em uma comunicação ou transação sejam quem dizem ser. Pode ser realizada por autenticação de usuário, como senhas, biometria ou tokens.

5. Não-repúdio

- Garantia de que uma ação ou transação não pode ser negada. Isso é frequentemente implementado com registros de auditoria, assinaturas digitais e logs de sistema que fornecem evidências claras de uma ação realizada.

6. Controle de Acesso

- Refere-se à gestão de permissões para acesso a sistemas, redes e dados. Pode ser baseado em diferentes modelos, como controle de acesso discricionário (DAC), controle de acesso obrigatório (MAC) ou controle de acesso baseado em função (RBAC).

7. Gestão de Riscos

- Envolve identificar, avaliar e mitigar riscos que podem afetar a segurança

das informações. Inclui a implementação de políticas e práticas para prevenir incidentes de segurança.

8. Proteção contra Ameaças

- Consiste em proteger as informações contra ameaças internas (como funcionários mal-intencionados) e externas (como hackers e malwares), usando ferramentas como firewalls, antivírus e sistemas de monitoramento.

Esses fundamentos formam a base para políticas, práticas e tecnologias de segurança da informação, garantindo que os dados e sistemas sejam protegidos contra diversos tipos de ameaças e vulnerabilidades.

Em ambientes ágeis, as ameaças à segurança da informação podem se intensificar devido à natureza flexível e colaborativa dos processos. Algumas das ameaças de segurança de tecnologia que podem acontecer nesses ambientes são:

1. Falta de controle sobre mudanças rápidas

Ambientes ágeis são conhecidos por mudanças rápidas no código e no design de sistemas. Isso pode resultar em **falta de revisão de segurança** em implementações recentes, deixando vulnerabilidades passarem despercebidas e expondo o sistema a ataques.

2. Desafios na gestão de acesso

A colaboração constante em equipes ágeis pode levar a uma **gestão inadequada de permissões e acessos**. Com muitas mudanças de membros de equipe e alterações de papéis, pode haver o risco de **acesso indevido a dados sensíveis** ou sistemas críticos.

3. Integração contínua e entrega contínua (CI/CD)

Embora as práticas de CI/CD melhorem a eficiência e a qualidade do desenvolvimento, elas também introduzem riscos de segurança. A automação de processos pode ser **mal configurada**, permitindo que código malicioso ou vulnerável seja introduzido rapidamente na produção, sem que haja uma verificação de segurança adequada.

4. Exposição a ameaças internas

Em equipes ágeis, onde a colaboração é intensa, **ameaças internas** podem ser mais difíceis de detectar. Isso inclui tanto ataques maliciosos quanto erros humanos por parte dos próprios membros da equipe, que podem comprometer a segurança do sistema sem querer.

5. Exposição de dados sensíveis

A velocidade das entregas em um ambiente ágil pode fazer com que as

informações sensíveis sejam compartilhadas de maneira inadequada entre os membros da equipe ou com partes externas, como fornecedores ou contratados. Isso pode resultar em **vazamento de dados** ou falhas na proteção de dados pessoais.

6. Falta de cultura de segurança

Em ambientes ágeis, onde a ênfase é muitas vezes colocada na rapidez das entregas e na inovação, a **segurança pode ser negligenciada**. A falta de uma cultura de segurança bem estabelecida pode resultar em lacunas de segurança não detectadas, como falhas no processo de codificação segura ou no uso de componentes vulneráveis.

7. Vulnerabilidades em ferramentas e dependências

A utilização de **ferramentas de colaboração** e **dependências externas** (como bibliotecas de código aberto) em ambientes ágeis pode introduzir riscos de segurança. Ferramentas mal configuradas ou dependências com falhas de segurança podem ser facilmente exploradas, afetando a integridade do produto final.

8. Escalabilidade de segurança

À medida que equipes ágeis crescem, é desafiador manter um **controle consistente de segurança** em toda a organização. Isso pode resultar em problemas como **falta de visibilidade** sobre as mudanças feitas no código e a introdução de novas vulnerabilidades sem uma revisão centralizada.

9. Ameaças externas

Ambientes ágeis, que frequentemente envolvem integração com terceiros (por exemplo, APIs externas ou fornecedores de software), podem ser vulneráveis a **ataques externos**. Esses terceiros podem ser alvos de ataques que afetam a segurança do sistema ágil sem que a equipe tenha controle sobre isso.

A **avaliação de riscos em ambientes ágeis** deve ser flexível e adaptativa para lidar com a natureza dinâmica e iterativa dos processos ágeis. Em ambientes como Scrum ou Kanban, onde mudanças rápidas e feedback contínuo são características predominantes, os riscos podem emergir a qualquer momento, exigindo uma abordagem proativa e constante de identificação e mitigação. Algumas etapas essenciais para realizar uma avaliação de riscos eficaz nesses ambientes:

1. Identificação de Riscos

A identificação de riscos é o primeiro passo e deve ser feita continuamente ao

longo de cada ciclo de desenvolvimento. Isso pode ser feito de várias formas:

- **Workshops de Riscos:** Em sprints de planejamento ou retrospectivas, envolva todos os membros da equipe para identificar potenciais riscos, tanto técnicos quanto organizacionais.
- **Revisões de código:** Avaliar o código frequentemente pode ajudar a identificar falhas de segurança ou qualidade que podem se tornar riscos no futuro.
- **Feedback das partes interessadas:** Envolver as partes interessadas e usuários finais pode revelar riscos relativos às expectativas do produto ou problemas de usabilidade.

2. Análise de Riscos

Uma vez identificados, os riscos devem ser avaliados com base em sua **probabilidade** de ocorrer e seu **impacto** caso se concretizem. No contexto ágil, esta análise precisa ser feita rapidamente e de forma colaborativa. Técnicas comuns incluem:

- **Matriz de Probabilidade e Impacto:** Classifique os riscos em uma matriz de 2x2 ou 3x3 para visualizar facilmente os mais críticos.
- **Análise qualitativa e quantitativa:** A análise qualitativa pode classificar os riscos de “alto”, “médio” e “baixo” impacto. Quando possível, use dados quantitativos (ex: custo estimado de uma falha) para priorizar os riscos.

3. Priorização de Riscos

Em um ambiente ágil, é importante priorizar os riscos com base na **gravidade** e na **probabilidade** de ocorrência. Isso ajuda a focar no que realmente pode afetar a entrega do projeto. As equipes podem usar ferramentas como:

- **Matriz de Riscos:** Priorizando os riscos com base em impacto e probabilidade.
- **Método MoSCoW:** Este método, utilizado em agilidade, pode ajudar a classificar os riscos em “Must have”, “Should have”, “Could have” e “Won't have”.

Garantir a **segurança em ambientes ágeis** exige a adoção de estratégias integradas, que considerem a natureza dinâmica e iterativa desses processos. A segurança precisa ser incorporada em todas as fases do ciclo de vida do desenvolvimento, sem comprometer a velocidade e a flexibilidade que são características das metodologias ágeis. Para garantir segurança podemos seguir algumas estratégias essenciais:

1. DevSecOps: Integração de segurança no processo ágil

O **DevSecOps** é uma abordagem que integra a segurança diretamente no ciclo de vida de desenvolvimento ágil. Em vez de tratar a segurança como uma etapa separada no final do desenvolvimento, ela se torna parte do fluxo contínuo de integração e entrega.

- **Automação de segurança:** Automatizar testes de segurança no pipeline de integração contínua (CI) e entrega contínua (CD) para identificar vulnerabilidades rapidamente.
- **Revisões automáticas de código:** Usar ferramentas de análise estática e dinâmica de código (SAST/DAST) para identificar falhas de segurança antes de o código ser implantado.
- **Monitoramento contínuo:** Implementar soluções que monitoram a segurança em tempo real, analisando tráfego, eventos e logs para detectar ameaças.

2. Treinamento e Conscientização sobre Segurança

Uma das formas mais eficazes de garantir a segurança é capacitar todos os membros da equipe sobre as melhores práticas de segurança.

- **Treinamento contínuo em segurança de TI:** Garantir que todos os desenvolvedores, gerentes e membros da equipe ágil recebam treinamento regular sobre vulnerabilidades de segurança (ex: SQL Injection, Cross-Site Scripting) e como evitá-las.
- **Cultura de segurança:** Estabelecer uma **cultura de segurança** em toda a organização, para que todos, desde desenvolvedores até stakeholders, compartilhem responsabilidades em garantir a segurança.
- **Workshops e sessões de conscientização:** Realizar workshops de segurança, simulações de ataques (como phishing) e revisões de código com foco em segurança.

3. Incorporar Práticas de Codificação Segura

A **codificação segura** é essencial para garantir que o código desenvolvido não introduza vulnerabilidades ou falhas de segurança.

- **Práticas de codificação segura:** Adotar diretrizes de codificação que evitem vulnerabilidades comuns, como injeção de SQL, XSS e falhas de autenticação.
- **Revisões de código:** Realizar revisões de código constantes, onde a segurança é um critério central, e as falhas de segurança são corrigidas antes do lançamento.
- **Ferramentas de análise de segurança no código:** Utilizar ferramentas como **Checkmarx** ou **SonarQube** para realizar análises estáticas e dinâmicas do

código e encontrar vulnerabilidades.

4. Gestão de Dependências e Bibliotecas Externas

O uso de bibliotecas externas e dependências de código aberto pode introduzir vulnerabilidades, pois elas podem não ser mantidas ou atualizadas adequadamente.

- **Gestão de dependências:** Usar ferramentas que verificam a **segurança** das bibliotecas e pacotes, como o **OWASP Dependency-Check** ou **Snyk**, para garantir que as dependências sejam seguras e estejam sempre atualizadas.
- **Evitar dependências desnecessárias:** Minimizar o número de bibliotecas externas e dependências, restringindo o uso apenas às que são estritamente necessárias.
- **Revisão e auditoria de código aberto:** Sempre revisar e auditar qualquer código aberto utilizado para garantir que ele não introduza vulnerabilidades.

5. Autenticação e Autorização Seguras

Garantir a **segurança de autenticação e autorização** é essencial para proteger dados e recursos em um ambiente ágil.

- **Autenticação multifator (MFA):** Implementar MFA para aumentar a segurança de processos críticos, especialmente para acessar ambientes de produção e dados sensíveis.
- **Controle de acesso baseado em funções (RBAC):** Adotar **RBAC** para garantir que os usuários tenham acesso apenas aos recursos necessários para suas funções, aplicando o princípio do **mínimo privilégio**.
- **Autenticação robusta:** Utilizar padrões modernos de autenticação, como **OAuth2** e **OpenID Connect**, para proteger o acesso ao sistema.

6. Testes de Segurança e Testes de Penetração

Incluir **testes de segurança** como parte integral do ciclo de desenvolvimento ágil pode ajudar a identificar vulnerabilidades antes que o sistema seja colocado em produção.

- **Testes de segurança contínuos:** Integrar testes de segurança no pipeline de CI/CD para detectar falhas e vulnerabilidades automaticamente.
- **Testes de penetração:** Realizar testes de penetração regulares para identificar pontos fracos que podem ser explorados por atacantes.
- **Testes de conformidade:** Verificar se a aplicação atende aos requisitos de segurança e conformidade (como GDPR, HIPAA) durante os testes.

7. Gerenciamento de Incidentes e Resposta Rápida

Mesmo com todas as medidas preventivas, incidentes de segurança podem

ocorrer. Ter uma **estratégia de resposta a incidentes** eficaz é essencial.

- **Plano de resposta a incidentes:** Ter um plano claro para identificar, mitigar e resolver incidentes de segurança rapidamente.
- **Simulações de incidentes:** Realizar simulações regulares de incidentes de segurança (como ataques de ransomware ou vazamento de dados) para treinar a equipe em como reagir rapidamente.
- **Análise pós-incidente:** Após um incidente, realizar uma análise para entender a causa raiz e evitar que se repita.

8. Monitoramento e Auditoria Contínuos

O monitoramento constante dos sistemas é crucial para detectar atividades suspeitas ou anômalas em tempo real.

- **Monitoramento em tempo real:** Usar ferramentas de monitoramento de segurança para detectar e responder rapidamente a ameaças em tempo real, como **Splunk** ou **ELK stack**.
- **Auditorias regulares:** Realizar auditorias de segurança e de conformidade de forma periódica, para identificar potenciais problemas antes que se tornem críticos.

9. Segurança nas Interfaces e APIs

As interfaces de programação de aplicativos (APIs) e as interfaces de usuário (UIs) podem ser pontos de entrada vulneráveis se não forem protegidas adequadamente.

- **Segurança de APIs:** Implementar **autenticação e autorização fortes** nas APIs e usar práticas como **rate limiting** e **criptação** para proteger dados sensíveis.
- **Validação e sanitização de entradas:** Validar e sanitizar todas as entradas de usuários para evitar ataques como **SQL Injection** e **Cross-Site Scripting (XSS)**.

10. Documentação de Segurança

A documentação de segurança é fundamental para garantir que todas as práticas e estratégias de segurança sejam compreendidas e seguidas por toda a equipe.

- **Documentar políticas de segurança:** Criar e manter documentos claros sobre políticas de segurança, incluindo padrões de codificação segura, controle de acesso e resposta a incidentes.
- **Acompanhamento de mudanças:** Manter um registro de todas as mudanças significativas nos sistemas e monitorar suas implicações para a segurança.

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

Conteúdo Bônus

Alguns filmes e documentários que podemos citar sobre segurança na tecnologia:

- Snowden de 2016, é um filme que fala sobre vazamento de informações da NSA que mostra questões de privacidade, vigilância em massa e ética na segurança da informação.
- O filme Blackhat de 2015, fala sobre ciberterrorismo e crimes digitais internacionais, e traz elementos técnicos realistas sobre ataques a sistemas industriais.
- Já o documentário Zero Days de 2016 fala sobre um ataque cibernético real contra o Irã e aborda ciberarmas, espionagem digital e temas como a guerra cibernética.

Referências Bibliográficas

STALLINGS, William. Segurança de Redes: Princípios e Práticas. São Paulo: Pearson, 2017.

PELTZER, Rolf Oppliger. Internet Security: Firewalls and Beyond. Boston: Artech House, 2010.

O que é segurança de TI? IBM. Disponível em <https://www.ibm.com/br-pt/topics/it-security>. Acesso em 10 de abril de 2025.

Riscos de segurança e tecnologia: como evitar na empresa? Equipe TOTVS, maio, 2024. Disponível em <https://www.totvs.com/blog/gestao-para-assinatura-de-documentos/riscos-seguranca-tecnologia/>. Acesso em 10 de abril de 2025.